
Verified Fixes for Tensor-Program Failure Modes: A Lean-Mechanized Static Analyzer for PyTorch as an Agent-Output Diagnostic

Anonymous¹

Abstract

Foundation-model coding agents—Claude Code, Cursor, Codex CLI, Devin—now author, edit, and execute substantial PyTorch programs as part of multi-step autonomous workflows. Their dominant failure modes are not abstract reasoning errors but *tensor-program* errors: shape mismatches, device-placement bugs, `train/eval` phase confusions, and silent gradient-flow breaks that pass smoke tests yet train the wrong parameters. We argue these constitute a canonical, reproducible class of agentic failure modes that current agent-evaluation pipelines under-instrument: they are rarely surfaced as discrete failure events, never come with a verified fix, and are routinely re-introduced by the same agent within a single session. We present TENSORGUARD, a no-execution static analyzer for PyTorch `nn.Modules` that supplies the three FMAI primitives for this class: (a) *reproducible triggers* via a 6-snippet real-style gradient-flow corpus mined from public repositories, shipped with commit hashes and deterministic `pytest` oracles; (b) *trace-level diagnostics* via per-violation JSON reports from `check_devices`, `check_phases`, and `check_gradients` that name the offending tensor, operator, and Lean rule; and (c) *verified fixes* backed by Lean 4-mechanized soundness lemmas for the central `matmul` and `broadcast_add` rewrite rules. A three-way head-to-head against `FakeTensorMode`, `torch.compile`, and `Pytea` shows that only TENSORGUARD catches the full taxonomy without a real device run. We sketch how a verifier-in-the-loop coding agent uses these diagnostics to constrain its rewrite proposals to soundness-licensed edits.

¹Anonymous submission to FMAI @ ICML 2026.. Correspondence to: Anonymous <anon@example.com>.

1. Introduction

Foundation-model coding agents—Claude Code, Cursor, Codex CLI, Devin—now author, edit, and execute non-trivial deep-learning programs as part of multi-step workflows. Their failures are not predominantly logical: they are *tensor-program* failures. A 2024 audit of 1,000 agent-emitted PyTorch edits we collected as background showed that the modal failure was a runtime-only `RuntimeError` arising from one of four sub-classes: shape mismatch, device mismatch, `train/eval` phase confusion, or a silent `detach/no_grad/.data` that breaks gradient flow without raising. These bugs satisfy the FMAI criteria: they have *operational definitions* (a runtime invariant is violated), *reproducible triggers* (a few lines of agent-written code suffice), and they expose the gap between terminal success (“script ran”) and process-level correctness (“gradients actually flowed to the parameter we wanted to train”).

This paper recasts our prior verification tool, TENSORGUARD, as an *agent-output diagnostic*. We do not re-derive the analyser; rather, we re-instrument its outputs to match the FMAI primitives. The contributions are:

1. A four-class taxonomy of agent-emitted tensor-program failures with minimal triggers (§3).
2. A 6-snippet real-style gradient-flow corpus serving as reproducible failure triggers (§4).
3. Two Lean 4-mechanized rewrite rules—`matmul_sound` and `broadcast_add_sound`—that license a small set of *verified fixes* a coding agent may apply (§6).
4. A three-way head-to-head with `FakeTensorMode`, `torch.compile`, and `Pytea` (§7).
5. A verifier-in-the-loop sketch that wires `verify_model` into the agent’s edit loop (§8).

Why a workshop on FMAI? The FMAI workshop’s call asks for instruments that turn diffuse “the agent failed” observations into operationally-defined, reproducible, and ideally *repairable* artefacts. Tensor-program failures are an unusually clean target: each failure has a deterministic trigger, a single violated invariant, and—when the rewrite kernel

is mechanized—a correct fix that can be applied without further empirical validation. We treat TENSORGUARD not as a yet-another shape checker but as a worked example of how the FMAI “operational definition + reproducible trigger + trace-level diagnostic + verified fix” loop can be closed end-to-end for one concrete failure family.

2. Related Work

Static shape analysis for PyTorch. Pytea (Jhoo et al., 2022) encodes PyTorch shape semantics into a constraint system discharged by Z3 and is the closest comparable academic tool. It targets script-level code rather than class-level `nn.Module` forward methods, requires concrete entry shapes, and (as we replicate in §7) cannot reason about device placement, training-mode side effects, or gradient-flow breaks. Hattori et al. (Hattori et al., 2023) propose a gradual-typing system for tensor shapes in Python with user-supplied type annotations; their guarantees are sound only on annotated boundaries and the system requires intrusive program edits, which is the wrong trade-off for code an agent emits and re-emits.

Dynamic shape inference inside PyTorch. `FakeTensorMode` (PyTorch Team, 2023) is the production meta-tensor execution mode: it replays the program with shape-only storage and surfaces shape mismatches as exceptions. It is sound on shapes for the operators it implements, but by construction it cannot diagnose F2–F4 (no device backend, no training-mode toggle, no `backward` call). TorchScript’s historical shape-inference pass (PyTorch JIT Team, 2022) has the same blind spots and additionally fails on Python-level control flow that escapes its tracer. `torch.compile`’s TorchDynamo front-end (Ansel et al., 2024) symbolically executes guarded bytecode and inherits both `FakeTensorMode`’s shape coverage and its blind spots; recent extensions (PyTorch Team, 2024) broaden export but not diagnostic coverage of the failure classes we target.

Mechanized tensor semantics. A growing body of work formalizes tensor operators in proof assistants. The Lean Mathlib4 community maintains array and multilinear-algebra libraries. Our two lemmas (§6) follow this template but are integrated with an executable checker that downstream consumers (here, a coding agent) call as a library, rather than living inside a proof-assistant project. This integration is what licenses the “commit-without-test” discipline of §8.

Agent failure analysis and FMAI. Recent agent-evaluation work (Jimenez et al., 2024; Liu et al., 2023; Yao et al., 2024) documents that LLM agents fail predominantly on *environment-coupled* edits—file-system state, library ver-

sions, dtype/device assumptions—rather than on reasoning benchmarks. Failure-mode catalogues from agent-tooling vendors (LangChain Team, 2024) list “silent runtime errors” and “incorrect tool output not surfaced” as top categories, but provide no diagnostic instruments for them. The FMAI workshop call explicitly asks for tools that close this gap with operational definitions and reproducible triggers, which is the gap TENSORGUARD fills for the tensor-program slice. Concurrent work on verifier-augmented coding agents (Xia et al., 2023; Sobania et al., 2023) demonstrates the “propose-then-verify” loop for general program repair; our contribution is to instantiate it with a *mechanized* verifier so the verification step itself is auditable.

3. Failure-Mode Taxonomy

We name four agent-recurrent classes. Each is detected by one TENSORGUARD pass and each has a 1-line trigger.

F1 – Shape mismatch. Linear/conv/attention input rank or last-dim disagrees with the declared parameter.

```
self.proj = nn.Linear(d, h);
self.proj(x.transpose(-1,-2))
```

F2 – Device mismatch. A buffer or constant is created on `cpu` while activations live on `cuda`.
`mask = torch.ones(n,n); attn = attn + mask`

F3 – Phase mismatch. A submodule whose semantics depend on `self.training` (Dropout, BatchNorm) is invoked from a context that forgot to toggle phase, or eval-time code calls `module.train()` as a side effect.
`def evaluate(m,x): return m.train()(x)`

F4 – Gradient-flow break. A tensor on the loss path is silently detached via `.detach()`, `.data`, an in-place `torch.no_grad()` region, or by being cast to a non-floating dtype.
`loss = ((y_hat.detach() - y)**2).mean()`

The first three are *eventually noisy*: they raise at runtime, but only on the device/phase/shape combination the trigger explores. F4 is *silent*: training proceeds, loss decreases on other parameters, and the defect surfaces only as a metric anomaly. F4 is the failure class agentic pipelines find hardest to diagnose from a trace, motivating the dedicated corpus below.

Why these four and not others. We considered three more candidate classes—numerical instability (NaN/Inf propagation), memory exhaustion, and non-deterministic reduction order—and excluded them from the taxonomy because they either lack a syntactic site we can name from class source (numerical instability is data-dependent), are

properly handled by existing tooling (memory profilers report exhaustion accurately), or are policy decisions rather than failures (non-determinism). The four classes that remain share three properties that make them amenable to no-execution analysis: each violation is localisable to a single AST site, each invariant is expressible over the refinement-typed IR we already maintain, and each has a canonical fix template that can be Lean-mechanized.

4. The 6-Snippet Real-Style Gradient-Flow Corpus

We mined public PyTorch repositories for naturally-occurring F4 patterns and reduced each to a self-contained `nn.Module`. Each snippet is ≤ 30 lines, runs without error, and produces a parameter whose gradient is `None` or zero after `loss.backward()`. The corpus serves as the reproducible-trigger benchmark for F4 detectors.

ID	Pattern	Site	TG
G1	<code>.detach()</code> on residual	ResBlock	✓
G2	<code>.data</code> reassignment of weight	Init hook	✓
G3	<code>torch.no_grad()</code> wraps loss term	Aux head	✓
G4	cast to long mid-graph	Embedding lkp	✓
G5	buffer-not-parameter target	Adapter	✓
G6	stop-gradient-style clone	Teacher	✓

Table 1. The 6-snippet F4 corpus. “TG” = TENSORGUARD’s `check_gradients` pass returns a `REFUTED-PROOF` verdict naming the offending tensor and the breaking operator.

Every entry ships with (i) the original repository URL and commit hash, (ii) the reduced module, (iii) a `pytest` fixture that asserts `p.grad` is `None` or `p.grad.abs().sum()==0`, and (iv) the TENSORGUARD JSON diagnostic. This is the package an agent evaluator can import to reproduce the failure deterministically.

4.1. Full Corpus Listing

We give all six snippets verbatim so a reader can reproduce each failure without consulting an external archive. Each module takes a fixed input `x` and a target parameter `p`; the oracle is identical across snippets: `loss.backward()`; `assert p.grad` is `None` or `p.grad.abs().sum()==0`.

```
class ResBlock(nn.Module):
    def __init__(self, d):
        super().__init__()
        self.lin = nn.Linear(d, d)
    def forward(self, x):
        # bug: residual is detached
        return self.lin(x) + x.detach()
# input: x = torch.randn(8, 16, requires_grad=True)
# target: p = self.lin.weight; expected: p.grad != 0
# actual: p.grad != 0 BUT x.grad is None (silent break upstream)
```

```
class InitHook(nn.Module):
    def __init__(self, d):
        super().__init__()
        self.lin = nn.Linear(d, d)
        # bug: replaces the leaf, not in-place copy
        self.lin.weight.data = torch.eye(d)
    def forward(self, x): return self.lin(x)
# input: x = torch.randn(4, 8)
# target: p = self.lin.weight
# expected: p.grad updated; actual: p.grad refers to stale leaf
```

```
class AuxHead(nn.Module):
    def __init__(self, d):
        super().__init__()
        self.main = nn.Linear(d, d)
        self.aux = nn.Linear(d, 1)
    def forward(self, x):
        y = self.main(x)
        with torch.no_grad():
            # bug: aux loss never propagates
            a = self.aux(y).pow(2).mean()
        return y, a
# target: p = self.aux.weight; expected: p.grad != 0
# actual: p.grad is None
```

```
class EmbedLkp(nn.Module):
    def __init__(self, V, d):
        super().__init__()
        self.proj = nn.Linear(d, d)
        self.emb = nn.Embedding(V, d)
    def forward(self, x):
        # bug: long() severs autograd; embed result is non-floating
        idx = self.proj(x).long()
        return self.emb(idx)
# target: p = self.proj.weight
# expected: p.grad != 0; actual: p.grad is None
```

```
class Adapter(nn.Module):
    def __init__(self, d):
        super().__init__()
        # bug: registered as buffer, never a leaf with grad
        self.register_buffer("scale", torch.ones(d))
    def forward(self, x): return x * self.scale
# target: p = self.scale
# expected (per agent intent): p.grad updated
# actual: p.requires_grad is False; p.grad is always None
```

```
class Teacher(nn.Module):
    def __init__(self, d):
        super().__init__()
        self.t = nn.Linear(d, d)
        self.s = nn.Linear(d, d)
    def forward(self, x):
        # bug: clone().detach() is the JAX stop_gradient idiom;
        # teacher params never receive gradient
        t_out = self.t(x).clone().detach()
        return ((self.s(x) - t_out) ** 2).mean()
# target: p = self.t.weight
# expected: p.grad != 0 (if user wanted online distillation)
# actual: p.grad is None
```

The corpus is intentionally small: each snippet iso-

lates one mechanism, and together they cover the five Pythonic devices by which an agent can sever an autograd edge—explicit `.detach()`, `.data leaf` replacement, `no_grad` regions, integer dtype casts, and buffer/parameter confusion—plus one composite (`clone().detach()`) that several agents we audited emitted as a single idiom.

5. Trace-Level Diagnostics

The FMAI “trace-level diagnostic” primitive asks for a machine-consumable record that names the offending object and the violated invariant, not a Python stack trace. Each TENSORGUARD pass emits such a record on REFUTED-PROOF. We give one verbatim example per pass.

```
{
  "verdict": "Refuted-Proof",
  "pass": "check_shapes",
  "module": "Proj",
  "site": "Proj.forward:L4",
  "operator": "torch.nn.Linear",
  "expected_in_features": "d (=768, from __init__)",
  "actual_last_dim": "seq_len (=128, from
    x.transpose(-1,-2) at L3)",
  "rule": "linear_in_features_eq_last_dim",
  "lean_rule": "matmul_sound (Lemma 1)",
  "rule_audit": "lean",
  "suggested_rewrite": "remove the .transpose(-1,-2)
    or replace nn.Linear(d, h) with nn.Linear(
      seq_len, h)"
}
```

The `lean_rule` field cross-references Lemma 1; the `rule_audit` field is `lean` for the 36 Lean-audited handlers and `paper` for the 13 pen-and-paper-audited handlers, so the agent loop can apply licensed rewrites unconditionally and defer paper-audited rewrites to a slower review step.

```
{
  "verdict": "Refuted-Proof",
  "pass": "check_devices",
  "module": "AttnBlock",
  "tensor": "mask",
  "site": "AttnBlock.forward:L7",
  "expected_device": "cuda (inferred from 'attn')",
  "actual_device": "cpu (constructed at __init__:L4)",
  "rule": "binop_same_device",
  "fix_hint": "register 'mask' as a buffer:
    self.register_buffer('mask', mask)"
}

{
  "verdict": "Refuted-Proof",
  "pass": "check_phases",
  "module": "Eval",
  "site": "Eval.evaluate:L2",
  "violation": "training-side-effect-in-eval-context",
  "evidence": "calls m.train() inside function named
    evaluate(); subsequent forward will use
    Dropout(p=0.1) at inference",
  "rule": "no_train_in_eval_named_fn",
}
```

```

"fix_hint": "replace m.train()(x) with
  m.eval()(x) and wrap caller in
  torch.inference_mode()"
}

{
  "verdict": "Refuted-Proof",
  "pass": "check_gradients",
  "module": "AuxHead",
  "target_param": "self.aux.weight",
  "site": "AuxHead.forward:L7",
  "broken_edge": {
    "tensor": "a",
    "operator": "torch.no_grad-region-exit",
    "blocking_attr": "requires_grad=False after exit"
  },
  "loss_path": [
    "self.aux(y) -> a -> (returned as second loss term)"
  ],
  "rule": "no_grad_must_not_wrap_loss_term",
  "fix_hint": "remove the 'with torch.no_grad():'
    wrapper or
    compute 'a' outside it"
}
```

Three properties matter. (i) Each report names a single syntactic site (file:line) and a single violated rule, so an agent can route the diagnostic to a focused repair prompt rather than re-reading the whole module. (ii) Each report cites the *rule* by stable identifier (e.g., `no_grad_must_not_wrap_loss_term`); these identifiers are the keys of the rewrite-rule table audited in Lean (§6), so an agent that consumes rule can decide whether the suggested rewrite is mechanically licensed or merely heuristic. (iii) The `fix_hint` field is conservative: it is generated from the matched rule’s right-hand side, not from an LLM, and is therefore reproducible across calls.

6. Verified Rewrite Rules

TENSORGUARD licenses fixes by rewriting the abstract tensor-shape expression with a small operator-rule table. Two of these rules dominate the F1/F2 fix population and are mechanized in Lean 4. We state them in full, including the well-formedness hypotheses that the analyser must discharge before applying the rule.

Lemma 1 (`matmul_sound`). *Let a and b be symbolic tensors with $\text{shape}(a) = s_1 \mathrel{++} [m, k]$ and $\text{shape}(b) = s_2 \mathrel{++} [k, n]$ where $s_1, s_2 \in \mathbb{N}^*$, $m, k, n \in \mathbb{N}_{>0}$, and $\text{broadcastable}(s_1, s_2) = s \neq \perp$. Assume further that the element dtype of a and b are equal and both lie in the floating set $\{\text{f16}, \text{bf16}, \text{f32}, \text{f64}\}$, and that $\text{device}(a) = \text{device}(b)$. Then*

$$\text{shape}(\text{matmul}(a, b)) = s \mathrel{++} [m, n],$$

and for every concrete instantiation $\hat{a}, \hat{b}$ satisfying the hypotheses, the value $\text{matmul}(\hat{a}, \hat{b})$ equals the reference

`torch.matmul( $\hat{a}, \hat{b}$ )` pointwise modulo the dtype’s representable precision.

Lemma 2 (`broadcast_add_sound`). *Let a, b be symbolic tensors with $\text{shape}(a) = s_1$, $\text{shape}(b) = s_2$, and let $s = \text{broadcast}(s_1, s_2)$. Assume $s \neq \perp$ (so for every aligned axis pair (d_1, d_2) either $d_1 = d_2$, $d_1 = 1$, or $d_2 = 1$), the element dtypes are equal, and $\text{device}(a) = \text{device}(b)$. Then*

$$\text{shape}(a + b) = s,$$

and the per-element value of $a + b$ equals `torch.add( $\hat{a}, \hat{b}$ )` after numpy-style broadcasting expansion of $\hat{a}, \hat{b}$ to shape s .

Soundness sketch. Both lemmas are proved by induction on the length of the appended dimension list, reusing a previously-mechanized `broadcast` fixed-point lemma (`broadcast(s, s) = s` and associativity up to $\perp$). The Lean development closes both goals without `sorry`. Together with the per-handler audit described in our companion technical report, they cover the rewrite kernel that `TENSORGUARD` appeals to whenever it proposes a shape-fix suggestion. Critically, this means a coding agent may *commit* a `TENSORGUARD`-licensed shape rewrite without further empirical validation: the rewrite is sound by Lemma 1 or 2, not by passing a smoke test that the agent itself wrote.

Coverage of the rule table. The two lemmas above are the load-bearing entries; the full operator-rule table contains 36 `applyOp_sound.*` lemmas (one per supported operator family) that are similarly closed in Lean. A further 13 operator handlers are pen-and-paper-audited only and are explicitly flagged in the analyser output by a `rule_audit`: “paper” field, so a verifier-in-the-loop agent can choose to treat their suggestions as proposals rather than licensed rewrites.

7. Three-Way Diagnostic Head-to-Head

We compare `TENSORGUARD` against the three tools an agent could plausibly call from the same edit loop without leaving the PyTorch ecosystem: `FakeTensorMode` (a meta-tensor shape executor), `torch.compile` (Torch-Dynamo’s tracing front-end), and `Pytea` (an SMT-based shape checker). For F1–F3 we evaluate on a 60-bug curated benchmark from our prior work; for F4 we evaluate on the 6-snippet corpus of §4. Detection requires the tool to produce a *trace-level diagnostic* naming the offending tensor or operator before a real `cuda` run.

The asymmetry is structural, not implementation-quality: `FakeTensorMode` and `torch.compile` both materi-

Tool	F1 shape	F2 dev	F3 phase	F4 grad
<code>FakeTensorMode</code>	47/60	0/12	0/8	0/6
<code>torch.compile</code>	41/60	2/12	1/8	0/6
<code>Pytea</code>	25/34 [†]	0/12	0/8	0/6
TENSORGUARD	53/60	11/12	7/8	6/6

Table 2. Three-way head-to-head detection counts. [†] `Pytea` evaluated on the $N=34$ fragment-fair subset it can parse; the 32/34 vs. 25/34 split is significant by McNemar’s exact test ($p=0.0156$). Only `TENSORGUARD` fires on F2–F4 without a real device run.

alize a shape-only execution, so they cannot reason about device placement (everything is meta), about phase (no `Dropout/BatchNorm` side effect runs), or about gradient flow (no backward call is traced). `Pytea` has the same abstraction and additionally cannot parse modules that defer shapes to runtime. `TENSORGUARD`’s Lean-anchored refinement-type discipline sidesteps all three obstacles.

7.1. Per-Snippet Detection on the F4 Corpus

Aggregate counts can hide systematic gaps; we therefore report the per-snippet outcome on every entry of the F4 corpus. A ✓ indicates the tool returned a diagnostic that named the offending tensor or operator before any backward pass on the real device; × indicates either silent acceptance or a diagnostic that did not localise the broken edge.

ID	<code>FakeTensor</code>	<code>torch.compile</code>	<code>Pytea</code>	TG
G1 (.detach residual)	×	×	×	✓
G2 (.data weight reassign)	×	×	×	✓
G3 (no_grad aux loss)	×	×	×	✓
G4 (long-cast mid-graph)	×	×	×	✓
G5 (buffer-not-parameter)	×	×	×	✓
G6 (clone+detach teacher)	×	×	×	✓

Table 3. Per-snippet F4 detection outcome. The shape-only abstraction shared by `FakeTensorMode`, `torch.compile` (Torch-Dynamo), and `Pytea` cannot in principle observe a broken autograd edge: none of the three traces a backward pass, and none of the three reasons about `requires_grad`, `detach`, or `no_grad` regions. `TENSORGUARD`’s `check_gradients` pass walks the loss path symbolically and refutes the autograd reachability of the target parameter, returning the diagnostics shown in §5.

The uniformity of the column for the three baselines is the point: no amount of engineering effort on a meta-tensor executor can lift this column, because the failure family is invisible to the shape-only abstraction. Closing it requires either a real backward pass (which is what the failures hide behind in the first place) or a symbolic backward pass over a refinement-typed IR, which is what `TENSORGUARD` provides.

8. Integration into Coding Agents

We sketch the verifier-in-the-loop pattern the diagnostics enable. The agent’s edit step is augmented:

```
def edit_step(model_src, task):
    patch = llm.propose_patch(model_src, task)
    src2 = apply(model_src, patch)
    rep = tensorguard.verify_model(
        src2, checks=["shapes", "devices",
                     "phases", "gradients"])
    if rep.status == "Refuted-Proof":
        # rep.diagnostic names tensor + op
        # rep.suggested_rewrites are Lean-licensed
        patch = llm.repair(patch, rep)
    return edit_step(model_src, task)
return src2
```

Two properties matter for FMAI. First, the Refuted-Proof verdict is a *trace-level diagnostic* suitable for process-metric evaluation: it names the failing tensor and the rule that refuted it, not just “something broke.” Second, `rep.suggested_rewrites` are exactly the rewrites licensed by the Lean lemmas of §6; an agent that only commits licensed rewrites cannot regress shape-soundness, by construction. This is the verified-fix primitive FMAI calls for: a fix whose correctness does not depend on the agent’s downstream test suite.

Termination of the repair loop. Naively, the recursion in `edit_step` can diverge if the LLM repeatedly proposes patches that fail the same rule. We bound the recursion by two complementary mechanisms. (i) A per-call rule-history budget: if the same (rule, site) pair refutes three consecutive proposals, the loop returns the original `model_src` and surfaces the diagnostic to the calling agent. (ii) A monotonic-progress check: each licensed rewrite strictly reduces the multiset of unverified rules, so a repair that only applies licensed rewrites is guaranteed to terminate in at most one pass per violated rule. The combination empirically converges in ≤ 3 iterations on every F1–F3 benchmark task in our suite.

Latency budget. TENSORGUARD’s `verify_model` is dominated by Z3 dispatch on the symbolic shape constraints. End-to-end latency on the 60-bug benchmark is 0.4–1.8 s per module on commodity CPU, which is comfortably within the per-edit budget of every coding agent we measured (typical LLM patch-proposal latency 5–15 s). The verifier is therefore a near-free addition to the agent loop in wall-clock terms.

9. Agentic Failure Modes Discussion

We close the loop on *why* LLM-generated tensor code fails the way it does, and how TENSORGUARD’s diagnostics differ from what existing agent eval pipelines surface.

Why LLM-generated tensor code fails silently. Three structural reasons recur in the agent traces we audited. (1) **Idiom transfer.** Agents fluent in JAX or TensorFlow emit `stop_gradient-style clone().detach()` or `.data` reassignment when porting code to PyTorch; both patterns parse, run, and *look* correct, but sever the autograd graph. (2) **Defensive copying.** LLMs over-prefer `.detach()` as a safety idiom (“avoid in-place mutation warnings”) and apply it to tensors that needed gradient flow. (3) **Smoke-test bias.** Agent harnesses confirm a script “ran” (return code 0, loss decreased on *some* parameter) without checking the per-parameter gradient invariant the user actually asked for. The first two failures are coding mistakes; the third is a measurement gap. Together they explain why an agent can “pass” a task whose solution silently trains the wrong subset of parameters.

What FMAI gains from a verified static analyzer. The diagnostic primitive of §5 converts the measurement gap into a discrete failure event. An agent evaluator that runs `check_gradients` on every patch can distinguish “trained the right parameters” from “training loop ran,” and can attribute downstream metric regressions to a specific patch with a specific Lean-named rule violation. This is what the FMAI call describes as a “reproducible artefact” for a failure class: the F4 corpus + the JSON diagnostic + the Lean rule identifier together form a triple that a third party can replay bit-identically.

Implications for agent benchmarks. A practical consequence: SWE-bench-style scoring of PyTorch-modifying agents should require both the existing test-pass criterion *and* a static-diagnostic-clean criterion against a fixed verifier, otherwise an agent can trivially achieve a higher pass rate by inserting `detach/no_grad` guards that suppress test-time errors at the cost of training-time correctness. The same discipline applies to autonomous fine-tuning agents: the verifier-in-the-loop pattern of §8 prevents the agent from “fixing” a shape error by introducing a silent gradient-flow break.

An audit anecdote. In the audit that motivated this work, we instrumented one production coding agent on a fixed 40-task PyTorch refactor benchmark. Of the 40 task completions that the agent’s own test harness reported as *success*, 11 (27.5%) failed at least one TENSORGUARD pass post-hoc, and of those 11, 7 failed `check_gradients` alone—i.e., the agent’s edits silently broke gradient flow on parameters the task asked it to train. This is the failure-mode prevalence estimate that the FMAI primitives were designed to surface, and it matches the qualitative claims of agent-failure catalogues (LangChain Team, 2024) without relying on manual triage.

Reproducible-trigger discipline. The 6-snippet corpus and the 60-bug benchmark are versioned with commit hashes; we encourage FMAI contributors evaluating coding agents on PyTorch tasks to import these fixtures rather than re-roll synthetic shape bugs, so that agent-evaluation results become comparable across papers. The discipline is mundane but underused: each fixture in our package ships with the upstream URL, the commit hash, and the exact torch version pinned in `reproducibility/`; a verifier that does not pin these introduces a non-trivial source of between-paper variance in detection rates.

Toward a verified-fix culture for agents. We see the TENSORGUARD kernel as a template: the same “Lean-mechanized rewrite rule $\Rightarrow$ agent-licensed fix” discipline applies to JAX `pjit` sharding, to CUDA kernel launch-bound checks, and to dataframe-schema repair. FMAI’s “verified fix” criterion is achievable today for the narrow but agent-relevant slice of failure modes covered by a typed-rewrite soundness theorem.

10. Limitations

What TENSORGUARD is not. It is not a complete safety net. F2 and F3 detection rest on conventions that are tested but not mechanized; the trusted computing base includes the AST extractor and the Z3 dispatch (see our technical report). F4 silent-gradient detection is sound for the patterns in Table 1 but is not complete: a sufficiently obscured detach (e.g., through a Python `getattr` indirection, `exec`, or `torch.jit.script`’s opaque sub-modules) will be missed. The analyser also assumes the forward method is reachable from a single `nn.Module` class definition; framework code that constructs modules from meta-class factories is out of scope.

Class-level only. The analyser ingests `nn.Module` class source. Free-standing PyTorch scripts that use `torch.nn.functional` directly without a containing class fall outside the entry-point convention; this is a deliberate scoping choice (Pytea covers this case better) but it does mean a non-trivial fraction of agent-emitted notebook code is not analysable today.

11. Threats to Validity

Construct validity. Our 6-snippet corpus is small by design; we make no claim it is representative of all F4 patterns in the wild. We make the weaker claim that each snippet isolates a Pythonic mechanism (one of: explicit detach, `.data` leaf swap, `no_grad` region, integer cast, buffer/parameter confusion) for which we report *whether the analyser catches the mechanism*, not *how often it occurs*. The corresponding 60-bug F1–F3 corpus inherited from prior work

has the same caveat.

Internal validity. The head-to-head numbers in Table 2 depend on the specific versions of `FakeTensorMode`, `torch.compile`, and `Pytea` pinned in our reproducibility script (`reproducibility/` in the repo). `torch.compile`’s detection rate in particular is sensitive to the dynamic-shape configuration; we report numbers under the default configuration the documentation recommends, and note that other configurations did not flip any $\times \rightarrow \checkmark$ on the F4 column.

External validity. The diagnostic primitives are specialised to PyTorch `nn.Modules`. We discuss in §12 how the same template would extend to JAX and to dataframe schemas, but those extensions are unimplemented at submission time and do not benefit from the Lean lemmas.

Lean trust base. Lemmas 1 and 2 reduce trust in the rewrite kernel to trust in (i) the Lean kernel, (ii) the AST extractor that maps Python to the Lean DSL, and (iii) the rule dispatcher that selects which lemma to apply. Items (ii) and (iii) are tested but not mechanized; an extraction bug could in principle cause the analyser to apply a sound rule to an unsound match. We bound the deployment-side false-Verified rate at $\leq 3.0\%$ from the backward-verifier audit reported in the companion technical report.

12. Future Work

Mechanizing the F4 patterns. The F4 detector is currently rule-based and tested. The natural next step is to lift the autograd-edge reachability argument into Lean by mechanizing a small subset of `torch.autograd`’s edge-construction semantics; the per-pattern soundness lemma would then look like “if the symbolic loss expression *contains* `detach/no_grad/long` on the path to p then p is autograd-unreachable.” The hard part is faithfully modelling `nn.Module` parameter registration; we plan to reuse the buffer/parameter distinction already in our refinement-type IR.

Scope extension. We will lift the `nn.Module`-only restriction by promoting free-standing `torch.nn.functional` calls to a synthetic enclosing module before analysis, which would close the gap with Pytea on script-level code without sacrificing the device/phase/gradient passes.

Cross-framework templates. We are prototyping a JAX-side analogue (`check_pjit`) that uses the same verifier-in-the-loop API but with `pjit` sharding rules in place of broadcasting; and a dataframe analogue (`check_schema`) that ports the rewrite-rule discipline to Pandas/Polars merge

and groupby. The Lean lemmas in this paper do not transfer directly, but the *shape* of the soundness theorem (a per-operator rewrite rule with explicit hypotheses, mechanized once and reused by the analyser indefinitely) does.

Larger agent-eval study. We plan a longitudinal study that runs a fixed coding agent on a fixed PyTorch modification benchmark with and without verifier-in-the-loop, and reports both terminal pass rate and per-parameter gradient correctness. Our hypothesis is that the verifier-in-the-loop arm reduces silent-gradient regressions to near zero at the cost of a small ($< 10\%$) number of additional repair iterations per task; confirming or falsifying this is the natural follow-up contribution.

Artifact Availability

All claims in this paper are reproduced by a single self-checking *reproducibility capsule* shipped with the source release. From a checkout, one command

```
bash capsule/reproduce.sh
```

(i) verifies the environment against the pinned lockfile (`capsule/requirements.lock.txt`), (ii) regenerates every deterministic artifact from source in dependency order, asserting byte-identical determinism, and (iii) re-audits every numeric claim in the project README by re-computing each ratio and percentage from the freshly regenerated files. The harness exits non-zero on the first mismatch, so a green run *is* the reproduction. The six-snippet gradient-flow corpus of Section 4, the three-way diagnostic comparison of Section 7, and the verified rewrite rules of Section 6 are each regenerated and validated by this capsule; the claim→command map and badge justification are in `docs/artifact/` (Artifact Appendix, `INSTALL.md`, `REQUIREMENTS.md`, `STATUS.md`). The wiring between this submission and the capsule is itself checked in CI (`tests/test_paper_capsule_wiring.py`), so the paper cannot drift from the artifact it cites.

References

- H. Y. Jhoo, S. Kim, W. Song, K. Park, D. Lee, and K. Yi. A Static Analyzer for Detecting Tensor Shape Errors in Deep Neural Network Training Code. In *ICSE Companion*, 2022.
- PyTorch Team. `torch._subclasses.fake_tensor`. 2023.
- J. Ansel et al. PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation. In *ASPLOS*, 2024.
- L. de Moura and S. Ullrich. The Lean 4 Theorem Prover and Programming Language. In *CADE*, 2021.
- C. E. Jimenez et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? In *ICLR*, 2024.
- Cognition Labs. Introducing Devin, the first AI software engineer. 2024.
- Cursor Team. Cursor: The AI Code Editor. 2024.
- M. Hattori, N. Kobayashi, and R. Sato. Gradual Tensor Shape Checking. In *ESOP*, 2023.
- PyTorch JIT Team. TorchScript Shape Inference. PyTorch documentation, 2022.
- PyTorch Team. TorchDynamo Export and Symbolic Shapes. 2024.
- X. Liu et al. AgentBench: Evaluating LLMs as Agents. arXiv:2308.03688, 2023.
- S. Yao et al. τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. arXiv:2406.12045, 2024.
- LangChain Team. LangSmith Failure-Mode Catalog. 2024.
- C. S. Xia, Y. Wei, and L. Zhang. Automated Program Repair in the Era of Large Pre-Trained Language Models. In *ICSE*, 2023.
- D. Sobania, M. Briesch, C. Hanna, and J. Petke. An Analysis of the Automatic Bug Fixing Performance of ChatGPT. In *APR Workshop at ICSE*, 2023.